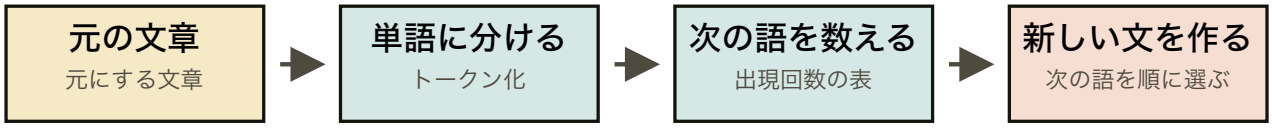
テキスト処理と文章生成

スクリプト言語演習

本日の到達点

- 手元の文章の特徴の集計
- 確率による次の語の選択と、文の生成
- **状態・表・ループ**による生成器の実装



この考え方の最初の例としてのマルコフ連鎖

第 1 回の道具の再利用

- 単語分割の出発点となる `split`
- 「次の語の回数」を記録する辞書
- 操作を後から追加する登録デコレータ

第 1 回で作ったもの	今回の使い道
`split` で文字列を分ける	単語に分ける出発点
辞書	「単語 → 次の語の回数」を記録
ディスパッチテーブル	「記号 → 実行する操作」を登録
登録デコレータ	操作をあとから追加

文字列の単語分割

- 空白分割で残る記号
- 正規表現による必要な文字列の抽出
- **字句解析**という作業

```
import re
```

```
text = "Hello, world! It's a nice day."

text.split()                          # 記号が残る
re.findall(r"[a-z']+", text.lower())
# ['hello', 'world', "it's", 'a', 'nice', 'day']
```

文章生成と電卓に共通する最初の処理

隣接する単語の集計

- 1 語の出現回数だけでは分からない続き
- 隣接する 2 語の集計による、次の語の把握
- 初出の語も 0 から集計する Counter

```
import re
from collections import Counter, defaultdict

text = "I like cats. I like dogs. I love cats."
words = re.findall(r"[a-z']+", text.lower())

following = defaultdict(Counter)

for a, b in zip(words, words[1:]):
    following[a][b] += 1

print(following["i"])  # Counter({'like': 2, 'love': 1})
```

3 行で作る「単語 → 次の語の回数」の表

文章生成と電卓の共通構造

- 機械が保持する現在の**状態**
- 次の処理を選ぶための**表**
- 状態を更新する**ループ**

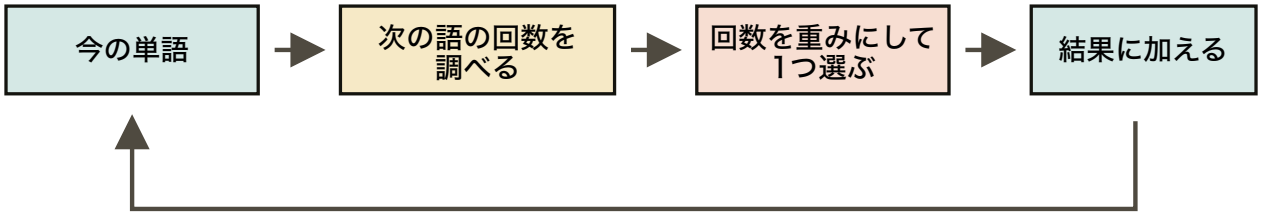
機械	状態	表に入れるもの	ループで行うこと
文章生成	今の単語	次に来た語と回数	次の語を選ぶ
電卓	数のスタック	記号と操作の関数	1 トークンを実行する

文章生成から電卓へ続く、共通の形

---

1 語だけの状態

- 覚えるのは現在の単語だけ
- 選んだ次の語を、新たな状態として利用
- 1 語でも残る、元の文章らしい特徴



直前の1語から、次の語を選ぶ

長い文脈の前に作る、1 語だけの生成器

次の語の回数表

- 隣接する 2 語を順に数え、辞書へ加算
- `table["i"]` に記録する「i の次の語」
- この例での like が 2/3、love が 1/3

```
from collections import Counter, defaultdict

def build_table(tokens):
    table = defaultdict(Counter)
    for a, b in zip(tokens, tokens[1:]):
        table[a][b] += 1
    return table

text = "I like cats. I like dogs. I love cats."
table = build_table(text.lower().replace(".", "").split())
# table["i"]    -> Counter({'like': 2, 'love': 1})
# table["like"] -> Counter({'cats': 1, 'dogs': 1})
print(table["i"])
```

回数による次の語の選択

- 等確率のため使わない `random.choice`
- 重み付きで選ぶ `random.choices`
- リストで返る値から取り出す末尾の `[0]`

```python
import random
from collections import Counter, defaultdict

def build_table(tokens):
    table = defaultdict(Counter)
    for a, b in zip(tokens, tokens[1:]):
        table[a][b] += 1
    return table

def pick_next(counter):
    words = list(counter.keys())
    weights = list(counter.values())
    return random.choices(words, weights=weights)[0]

text = "I like cats. I like dogs. I love cats."
table = build_table(text.lower().replace(".", "").split())
print(pick_next(table["i"]))    # 'like' が出やすく、'love' がたまに出る
```

回数の重みに残る、元の文章の頻度

状態更新による文章生成

- 現在の語という**状態**と、table という**表**
- 状態を更新する for という**ループ**
- 次の語が表にない場合の終了

```python
import random
from collections import Counter, defaultdict

def build_table(tokens):
    table = defaultdict(Counter)
    for a, b in zip(tokens, tokens[1:]):
        table[a][b] += 1
    return table

def pick_next(counter):
    words = list(counter.keys())
    weights = list(counter.values())
    return random.choices(words, weights=weights)[0]

def generate(table, start, n=30):
    word = start
    result = [word]
```

```
    for _ in range(n):
        if word not in table:
            break
        word = pick_next(table[word])
        result.append(word)
    return " ".join(result)

text = "I like cats. I like dogs. I love cats."
table = build_table(text.lower().replace(".", "").split())
print(generate(table, "i"))  # 例: "i like cats"
```

温度による意外さの調整

- 重みを 1/T 乗してから行う次の語の選択
- 低温での最頻語への偏りと、高温でのばらつき
- T = 1 における元の統計

```
from collections import Counter

counter = Counter({"like": 3, "love": 1})
temperature = 0.8
weights = [c ** (1 / temperature) for c in counter.values()]
print(weights)  # 温度で重みを調整した例
```

温度	重み (3, 1)	出力の傾向
0.1	(59049, 1)	最頻語に偏る。安定するが単調
1.0	(3, 1)	元の統計どおり
5.0	(1.25, 1)	ほぼ等確率。多様だが崩れやすい

> モデルの作り直しが不要な、確率のつまみとしての温度

文脈の長さと表の大きさ

- 直前 2 語による自然な出力
- 見る語数とともに急増する組み合わせ
- 長い文脈で増える、表にない並び

見る語数	出力の傾向	問題
1 語 (bigram)	単語のつながりは自然	文の意味は崩れやすい

見る語数	出力の傾向	問題
2 語 (trigram)	かなり自然	元の文章を写しやすい
5 語	ほぼ丸写し	組み合わせが表に無くなる

第 3 回で扱う、表を学習済みの関数に置き換えた言語モデル

逆ポーランド記法

- 不要な括弧と優先順位
- 数の push と、演算子での pop・計算
- そのままスタックとなる Python のリスト

読んだもの	やること	スタック
`3`	数なので push	`[3]`
`4`	数なので push	`[3, 4]`
`+`	2 つ pop して足し、結果を push	`[7]`

`10 3 -` の結果は 7。先に pop される値は 3

表に登録する操作関数

- 「単語 → 次の候補」から「記号 → 関数」への表の置換
- 第 1 回の登録デコレータを使う `@word`
- ワードを追加しても変わらない評価の骨格

```
WORDS = {}

def word(name):
    def deco(func):
        WORDS[name] = func
        return func
    return deco

@word("+")
def _(stack):
    b, a = stack.pop(), stack.pop()
    stack.append(a + b)
```

引き算と割り算で重要な、取り出す順序

拡張可能な評価ループ

- トークンへの分割と、表内の関数の呼び出し
- 表にないトークンの数としての push
- 操作を増やしても変わらないループ

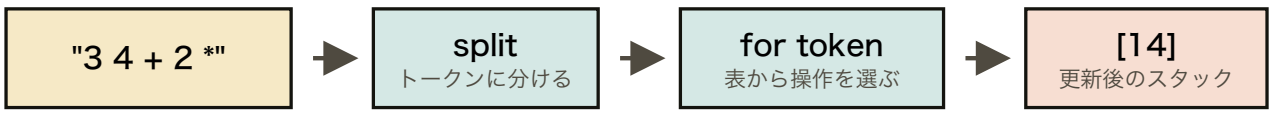
```
WORDS = {}

def word(name):
    def deco(func):
        WORDS[name] = func
        return func
    return deco

@word("+")
def _(stack):
    b, a = stack.pop(), stack.pop()
    stack.append(a + b)

def evaluate(stack, text):
    for token in text.split():
        if token in WORDS:
            WORDS[token](stack)
        else:
            stack.append(int(token))

stack = []
evaluate(stack, "3 4 +")
print(stack)  # [7]
```



---

2 つの機械の共通骨格

- 状態・表・ループという 3 要素
- 語を選ぶ文章生成と、関数を実行する電卓
- 両方の骨格となる**辞書とループ**

	文章を作る機械	計算する機械
状態	今の単語	数のスタック
表	次の語の回数	記号と操作
更新	確率に従って選ぶ	関数を実行する

---

with によるファイル読み込み

- ファイル全体を 1 つの文字列にする read()
- 大きいファイルに対する行ごとの処理
- 日本語で指定する encoding="utf-8"

```
from pathlib import Path

# サンプルファイルを用意（既存なら上書きしない例でも可）
Path("data.txt").write_text("hello\n# comment\nworld\n", encoding="utf-8")

with open("data.txt", encoding="utf-8") as f:
    text = f.read()
print(text)

with open("data.txt", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        print(line)
```

省略時に環境によって起こる文字化け

正規表現による必要部分の抽出

- r"..." で記述するパターン
- 全件抽出の findall と、置換の sub
- マッチしない場合の match の None

記法	意味
.	任意の 1 文字
\d \w \s	数字 / 英数字と _ / 空白

記法	意味
`[abc]` `[^abc]`	いずれか / それ以外
`*` `+` `?`	0 回以上 / 1 回以上 / 0 か 1 回
`( )`	グループ。取り出したい部分

```python
import re

text = "Hello 123 world"
s = "2026-08-24"
print(re.findall(r"[a-z']+", text.lower()))
print(re.sub(r"\d+", "N", text))
m = re.match(r"(\d{4})-(\d{2})", s)
if m:
    year, month = m.groups()
    print(year, month)
```

並べ替えとスタック操作

- 回数の降順と、同数時の辞書順
- スタック上部を操作する DUP と SWAP
- 電卓の課題で登録するワード

```python
from collections import Counter

counts = Counter({"apple": 3, "banana": 2, "cherry": 3})
print(sorted(counts.items(), key=lambda kv: (-kv[1], kv[0])))
# [('apple', 3), ('cherry', 3), ('banana', 2)]
```

ワード	動作	スタックの変化
DUP	一番上を複製	(a → a a)
DROP	一番上を捨てる	(a →)
SWAP	上 2 つを入れ替え	(a b → b a)
OVER	2 番目を上にコピー	(a b → a b a)
ROT	3 番目を一番上へ	(a b c → b c a)

前の課題の結果の利用

- 課題 1〜3 で完成する、語を状態にする機械

- 課題 4 で行う、同じ骨格のままのスタックへの状態の置換
- 温度と 2 語文脈による出力の比較という発展課題

#	課題	ねらい
1	文章をトークンに分ける	ファイル入出力、正規表現
2	「次に来る語」の表を作る	`defaultdict`、`Counter`、`zip`
3	表を使って新しい一文を作る	重み付き抽選、生成ループ
4	表の値を操作に替え、電卓を動かす	スタック、ディスパッチ

課題の進め方

- 「Colab で開く」から課題を開き、「ドライブにコピーを保存」
- 準備セルの実行と、各課題の `TODO`
- 確認セルの保存
  - 教員は別のテストケースで採点するため、削除しても採点結果は不変

面白い出力や自作ワードの共有